

Paperplane: zero to mastery

Docling, PDF Inspector, cloud AI, and local Ollama document intelligence

Table of Contents

1. Start	3
2. Parse	3
3. Organize	3
4. Understand grounding	3
5. Jobs and privacy	3
6. Follow the code	3
7. Verify	4

1. Start

Double-click Paperplane.cmd on Windows or run ./Paperplane.sh on Linux. Each launcher installs only missing or out-of-date supported prerequisites and opens the multipage Streamlit app at http://127.0.0.1:8551. Once ready, later launches skip setup. Manual entrypoint:

```
uv run --locked --extra cpu streamlit run workspace_app.py --server.port=8551
```

2. Parse

All four engine toggles start off. Choose Docling ADE, PDF Inspector ADE, Cloud AI ADE, or Ollama ADE. Optionally add cloud enhancement to a local engine. Upload up to 20 files, choose each one-based inclusive page range, then parse. Files run concurrently but remain context-isolated.

GLM-OCR, PaddleOCR-VL, and DeepSeek-OCR use a local CPU PP-DocLayoutV3 detector. Ollama recognizes each region with its family-native prompt, and Paperplane uses detector boxes for candidate grounding. RapidOCR contributes only exact final word-box alignment. DeepSeek retries one empty or transiently failed text crop. An isolated exhausted region is reported as a warning while successful regions remain; three consecutive failures stop the page and surface an actionable Ollama error.

All Parse controls sit below navigation in the sidebar. One main document selector drives Input preview, Output, Annotated PDF, Markdown, HTML, and JSON. Download strict ADE v2 JSON when you need the compatibility-shaped hierarchy; download Paperplane JSON for observed words, confidence state, provenance, warnings, and document relations. Standalone HTML is sanitized. The batch ZIP groups every successful document's available outputs and records failures in a versioned manifest without copying source uploads.

3. Organize

Organize runs Classify, Split, and Section over the grounded Parse response. Results retain source ranges and identify deterministic partials.

4. Understand grounding

Pages and blocks carry normalized boxes and half-open Unicode ranges. Text/marginalia use atomic line grounding; table cells carry row/column/span data. Native PDF words and RapidOCR word observations are emitted only when their text aligns exactly. IDs are zero-based and response-local in strict ADE v2 output.

5. Jobs and privacy

SQLite metadata and private artifacts live under %LOCALAPPDATA%\Paperplane for seven days. Jobs exposes status, cancellation state, per-job deletion, and clear-all. Credentials are never stored there. Cloud engines, including Agnes, transmit only selected pages. Agnes sends page PNGs inline instead of publishing them at public URLs. Ollama, Docling, PDF Inspector, and storage remain local.

6. Follow the code

1. workspace_app.py — navigation.
2. streamlit_app.py and app_pages/ — UI flows.
3. paperplane/runtime.py — batch/provider composition.
4. paperplane/parser.py — range and page orchestration.
5. paperplane/ollama_document.py and ollama_ocr.py — Ollama discovery, layout regions,

prompts, and crop recognition.

6. `paperplane/contracts.py` and `ade_contracts.py` — internal and public contracts.
7. `paperplane/ade_workflows.py` — cited workflows.
8. `paperplane/jobs.py` — durable lifecycle.
9. `paperplane/outputs.py` — sanitized HTML and safe batch archive assembly.
10. `document_intelligence.py`, `calibration.py`, `benchmark.py` — semantic and evaluation layers.

7. Verify

```
uv run ruff check paperplane app_pages tests streamlit_app.py workspace_app.py scripts
uv run pyright
uv run pytest -q
uv run python scripts/benchmark_report.py
```

The initial benchmark corpus is an integrity baseline, not an accuracy claim. Publish raw outputs, prompts, versions, costs, failures, and calibration provenance before publishing scores.